PodLove Final Specification

0. Global Rules (Applies to ALL Six Sections)

- Scope lock: This document is the complete requirements list. If a requirement appears anywhere in this document, it may not be labeled "new" at delivery time.
- Screens are not features: A feature is complete only when the screen exists AND the backend logic, rules, validations, timers, permissions, and integrations behave exactly as specified across iOS, Android, and Web.
- Server-side enforcement: Subscription entitlements, role permissions, chat timers, and matching eligibility MUST be enforced on the backend. Front-end-only enforcement is not acceptable.
- Parity rule: Mobile and Web must mirror each other in functionality, copy, required fields, validations, and user flows unless an explicit exception is written in this document.
- Minimum viable episode rule: PodLove must NEVER create an episode lineup with only two people. Minimum is 3 total participants: 1 Spotlight + at least 2 Sparks.
- Auditability: Admin must be able to see logs for matching decisions, scheduling actions, reminder delivery, and key lifecycle state changes.
- QA + acceptance: Developers complete internal QA, deliver a build, Naketa tests and returns one consolidated bug list (with screenshots), developers fix all listed issues. Project is complete after fixes unless critical functional failures exist.

1. Mobile Apps (iOS + Android)

Goal: Provide the complete end-user PodLove experience on mobile, including onboarding, subscriptions, matching participation, podcast participation, surveys, notifications, and chat (as permitted).

1.1 Core User Flows (Must Work End-to-End)

- Sign Up / Sign In / Password Reset
- Full onboarding completion (all required fields)
- Subscription selection and purchase (Sampler, Seeker, Scout)
- Profile view + edit (with validations)
- Role assignment notifications (Spotlight vs Spark)
- Episode 1 participation flow (Green Room → Live Room)
- Episode 2 participation flow (chosen pair)
- Surveys: Sign-up, 1st-Date, Episode 2 outcome
- Chat access (only when permitted by Chat Rules section)

PodLove Final Specification

1.1A Account Deletion and Pool Removal (Required)

Users may delete their account from the application.

Upon account deletion, the system must:

- Immediately set user_active = false
- Remove the user from all matching eligibility and any current or future episode assignments
- Prevent all future reminders, surveys, and chat access from triggering for the deleted user
- Log the deletion event (user_id, timestamp, and action) for audit purposes

Data Retention:

Survey and match logs may be retained in anonymized form for system analytics and AI improvement unless prohibited by policy. Deleted users must not be reintroduced into the matching pool or contacted by the system.

1.2 Mobile UI Requirements (Functional, Not Visual-Only)

- All screens must be fully wired to backend logic (no static placeholders).
- All subscription restrictions must be reflected in UI AND enforced server-side.
- All date/time displays must use the user's local timezone and show clear time + date.
- All timers (chat windows, deadlines) must be accurate to the hour.

1.3 Notifications on Mobile

- System notifications must trigger for: role assignment, episode scheduling confirmation, reminder notifications, survey prompts, and chat open/close events (where applicable).
- Podcast reminders: the system must send two reminders after scheduled episode time: (1) 1 week before, (2) 48–24 hours before.
- If push notifications are not available for a user/device, the system must fall back to email notifications (same timing).

2. Web App (Responsive Website With Identical Features)

Goal: The website must be an operational duplicate of the mobile experience for all user-facing functionality.

PodLove Final Specification

2.1 Parity Requirements (Non-Negotiable)

- Web must mirror Mobile: onboarding, subscriptions, profile fields, match participation, booking flows visibility, surveys, chat access rules, role notifications, and residency messaging.
- If Mobile supports a feature, Web must support the same feature unless an explicit exception is written here.
- Copy, required fields, validations, and entitlements must be identical.

2.2 Web Authentication and Account Management

- Sign Up / Sign In / Password Reset must function identically and be visually identical to mobile.
- Account security: sessions, logout, and password reset must be robust and reliable.

2.3 Web Notifications

- All email triggers must follow the same rule set as mobile.
- Podcast reminders must follow the same two-reminder rule (1 week and 48–24 hours before).

3. AI Matching Engine (Batch-Based Matching System)

Goal: Create correct episode lineups (Spotlight + Sparks) using OpenAI-based compatibility scoring AND rotation/fairness rules. This system must prevent instant matching on signup and must prevent users waiting indefinitely.

3.1 Non-Negotiable Matching Principles

- Batch-based matching only. Users are NOT matched on signup.
- AI waits until there is a viable pool before creating Episode 1 lineups.
- The first two users must **NEVER** be auto-matched due to signup order or lack of pool; matching must **NOT** trigger from early signups.
- Sparks are matched ONLY to a Spotlight. Sparks are **NEVER** matched to each other.
- Minimum viable episode rule: must **NEVER** create a lineup with fewer than 2 Sparks (i.e., fewer than 3 total participants).
- Matching quality is prioritized over speed (do not relax rules silently).
- Admin MUST be able to view all matches AI created.

3.2 Eligibility Gate (No Matching Until Eligible)

- A user becomes eligible only after completing: full onboarding, questionnaire, preferences, residency city selection, and subscription selection.

- Users missing any required step must be **excluded** from matching **AND** prompted to complete their profile.
- Users with user_active = false (including deleted accounts) must be excluded from all matching and episode assignment processes.
- Enforce 35–55 age range and user-set age preferences (two-way compatibility).

3.3 Batch Trigger Rules

- Matching runs when the pool is viable and system is preparing an Episode 1 lineup.
- Pool viability must be checked BEFORE any OpenAI scoring calls are executed.
- Pool viability must ensure at least: 1 eligible Spotlight candidate and at least 2 eligible Sparks (depending on subscription plan) for that Spotlight.
- System must list **EVERY** user in dashboard

3.4 Role Assignment: Spotlight and Spark

The AI Matching Engine is responsible for assigning roles for each Episode 1.

- For every Episode 1 created, the AI must:
  - Select **one eligible user to serve as the Spotlight**
  - Select the required number of eligible users to serve as Sparks (2, 3, or 4 based on subscription tier)
- Role assignment must occur **as part of the batch matching process**, not before and not after.

Eligibility rules:

- Only users marked as user_active = true and meeting eligibility requirements may be assigned roles.
- A user may NOT be assigned as both Spotlight and Spark in the same episode.
- A user may not be assigned to more than one Episode 1 at a time.

Authority rule:

- The AI is the sole authority for role assignment.
- Admin may view assigned roles but does not approve or override them except in exceptional, manual intervention cases.

3.4A Episode 1 Match Selection and Chosen Spark Authority

Chosen Spark and Consent Authority

PodLove Final Specification

The Spotlight selects one Spark during the Episode 1 recording session. This selected person is stored as chosen_spark_user_id and is the single source of truth for which Spark was selected.

After the recording, the selected Spark must explicitly confirm whether they agree to proceed. This is stored as chosen_spark_accepted (true/false).

The system must not infer the chosen Spark from chat activity, surveys, AI scoring, or any other signal. The system must not infer consent from any signal.

Automation rules:

- Chat access must open only when Episode 1 is marked Completed AND chosen_spark_user_id is set AND chosen_spark_accepted = true.
- If chosen_spark_accepted = false, chat must not open and Episode 2 must not be scheduled. The episode outcome must be recorded as "No Mutual Match."
- Unchosen Sparks must never receive chat access.

chosen_spark_user_id (and chosen_spark_accepted when true) is used to:

- Determine which two users are eligible for chat (Spotlight + chosen Spark)
- Determine Episode 2 participant roster (Spotlight + chosen Spark)
- Determine who receives pair-only post-date and Episode 2 outcome surveys

Log match outcome for AI learning (chosen, accepted/declined, and subsequent outcomes)

3.5 OpenAI Compatibility Scoring (Required)

- OpenAI MUST be used to compute compatibility between Spotlight and each candidate Spark.
- OpenAI integration must run on PodLove's own production keys (environment variables).
- For each Spotlight–Spark candidate pair, call OpenAI with structured profile + questionnaire data (and, over time, learned survey feedback).
- OpenAI must run (minimum): compatibility_score (0–100) plus a short explanation. Component scores (values/lifestyle/conflict-style) are recommended.

3.5B Question Importance Weighting and Dealbreaker Weighting

Purpose: Matching quality must be driven primarily by high-importance items.

PodLove Final Specification

3.5B.1 Required Data Model Fields

Every questionnaire item must include and store the following fields:

- question_id (unique)
- question_text
- answer_format (e.g., single select, multi select, scale, free text)
- user_answer (stored per user, per question)
- importance_weight (integer, required)

Importance scale (authoritative):

- 1 = Low importance (nice-to-have / preference)
- 2 = Medium importance
- 3 = High importance (core alignment)
- 4 = Critical importance (dealbreaker-level importance)

3.5B.2 Definition of "Dealbreakers" in This System

- Any question with importance_weight = 4 is treated as **dealbreaker-level importance**.
- Misalignment on one or more importance_weight = 4 items must strongly reduce compatibility scoring.
- Alignment on many low-importance questions must **not** compensate for misalignment on critical-importance questions.

3.5B.3 Required OpenAI Input Format (Per Candidate Pair)

For each Spotlight–Spark candidate comparison, the OpenAI request must include the full list of question/answer pairs with weights:

For each question:

- question_id
- question_text
- importance_weight
- spotlight_answer
- spark_answer

Developers must pass the weights into the OpenAI prompt/message so the model can correctly prioritize high-importance questions.

3.5B.4 Required Scoring Behavior

The compatibility scoring logic must enforce the following:

- High-importance questions (importance_weight 3–4) must have greater influence on the overall compatibility result than low-importance questions (importance_weight 1–2).
- Multiple mismatches on importance_weight = 4 questions must result in a **low compatibility_score** and must generally disqualify the candidate from selection (see minimum threshold rules in Section 3.4A).

A candidate who aligns on only a small number of low-importance questions (e.g., "3 out of 100") must not be treated as a match and must return a low compatibility_score.

3.5C Compatibility Acceptance Thresholds

Compatibility scores generated by OpenAI must meet minimum acceptance criteria before a Spark may be selected for an Episode 1 lineup.

Acceptance rules:

- **Hard disqualifier:** Any mismatch on questions with importance_weight = 4 that are marked as non-negotiable for the Spotlight disqualifies the candidate Spark. Admin override is permitted with logging.
- **Minimum score threshold:** A candidate Spark must receive a compatibility score of **70/100 or higher** to be eligible for selection.
- **Insufficient pool handling:** If the eligible pool does not contain threshold-compliant candidates, the system must not silently lower thresholds. The system must either:
    - Defer matching until threshold-compliant candidates are available, or
    - Flag "Insufficient pool" in the Admin Dashboard.

Threshold rules must be enforced server-side.

3.6 Pool, Waiting-Time, and Rotation Rules

- Time-in-pool must be tracked to prevent indefinite waiting.
- Rotation logic must prevent the same people from always being Sparks.
- Duplicate pair prevention: avoid pairing the same two people repeatedly unless an explicit plan entitlement exists (second-chance / refresh) or admin override is used.

- Spark saturation: implement safeguards to prevent overusing a single Spark in a short period.
- Never assign Sparks based on signup order.

3.7 AI Learning Loop (Must Improve Over Time)

- AI must improve recommendations using feedback from: Episode 1 survey, Post-Date survey, and Episode 2 outcome survey.
- Developers must store survey responses in a way that can be used by OpenAI in future matching calls.
- **OpenAI must not be static or random; it must learn from feedback.**

3.7A AI Learning Loop – Minimum Technical Requirements

To ensure the AI matching system meaningfully improves over time (and does not remain static), the following minimum technical requirements must be implemented:

- Store all survey responses with user_id, episode_id, timestamp, and survey_type.
- For each new OpenAI compatibility scoring call, include (at minimum) the user's most recent:
    - Survey 1
    - Survey 2
    - Episode 2 outcome summary (if available)
- Log the OpenAI input payload version (schema version) and the output (compatibility score and explanation) for each match for auditability.
- The system must be capable of incorporating stored feedback into future OpenAI compatibility scoring calls, not merely storing survey data without reuse.

OpenAI usage must NOT be static or random. The learning loop must demonstrate the ability to evolve scoring inputs over time based on accumulated feedback.

3.8 State Integrity (No Stuck Users)

- Matching system must maintain consistent user states so users cannot be stuck forever in assigned states.
- If a user becomes inactive or unavailable, the system must remove them from eligibility and release any future assignments.

3.9 Episode Lifecycle States and Automation Triggers

PodLove Final Specification

To prevent incorrect automation, stuck users, and invalid AI learning data, the system must implement explicit lifecycle states for Episode 1.

Episode 1 States:

- **Drafted** – Episode lineup created by AI; not yet scheduled
- **Scheduled** – Admin has set official recording date/time
- **In-Progress** – Recording room opened
- **Recorded** – Recording completed successfully
- **Completed** – Admin confirms episode completion and outcome
- **Canceled** – Episode will not occur
- **Failed** – Episode attempted but unusable (technical failure or insufficient attendance)

Automation Triggers:

- Reminder notifications may only be sent while the episode is in **Scheduled** state.
- Chat access may open only when the episode transitions to **Completed** AND the Chosen Spark is set AND mutual consent is confirmed.
- Episode 1 surveys may be sent only after the episode transitions to **Completed**.
- If an episode is marked **Canceled** or **Failed**:
    - The episode lineup is voided
    - No chat opens
    - No surveys are sent
    - All participants are returned to the Eligible Pool

State transitions must be enforced server-side and must not be inferred from time passage alone.

3.9A Episode 1 States: Drafted (created by AI) ↠ Scheduled (admin sets date/time) ↠ In-Progress (room opened) ↠ Recorded (recording completed successfully) ↠ Completed (admin confirms completion) OR Canceled (will not occur) OR Failed (attempted but not usable; reschedule required).
Triggers:

- Reminders only send in Scheduled state.
- Chat opens only when Episode 1 becomes Completed AND Chosen Spark is set.
- Episode 1 surveys send only when Episode 1 becomes Completed.
- If Episode 1 is Canceled/Failed, all participants return to Eligible Pool and the episode lineup is voided.

3.9.B Attendance and No-Show Handling Rules

PodLove Final Specification

Episode 1 may proceed only if the Spotlight attends and at least two Sparks attend.

Attendance rules:

- If the Spotlight does not attend, the episode must be marked **Canceled**.
- If one Spark does not attend and at least two Sparks remain, the episode may proceed.
- If attendance drops below two Sparks, the episode must be marked **Failed**.

For episodes marked **Canceled** or **Failed**:

- The episode lineup is voided.
- No chat access opens.
- No surveys are sent.
- All participants are returned to the Eligible Pool.

Attendance determination and state marking are performed by Admin and enforced server-side.

4. Podcast Platform (100ms)

Goal: Provide reliable Episode 1 and Episode 2 recording environments with correct roles, permissions, views, and recordings. The behavior must match PodLove's room/role rules every time.

4.1 Room Types

- Green Room: Producer sees all cameras/mics; guests cannot see Producer; Producer verifies readiness; no recording allowed.
- Live Room (Episode 1): Spotlight, Sparks, Host visible; Producer hidden but present; multi-track recording enabled; Spotlight View + Grid View modes; landscape view locked.
- Episode 2 Room: Host + Spotlight + chosen Spark; same technical behavior and producer supervision.

4.2 Role Permissions

- Producer: hidden from participants; producer can see all feeds; admit/remove participants; mute/unmute; start/stop recording; manage on/off stage; communicate by text to host/guests as needed.
- Host: always visible; guides the conversation; must have camera/mic enabled.
- Host can admit/remove participants; mute/unmute; start/stop recording;

- Spotlight + Sparks: visible participants per episode rules; cannot access producer controls.

4.3 View Modes and Speaker Highlighting

- Spotlight View must exist as an option: active speaker appears larger; others smaller.
- Grid View must exist as an option: all participants equal size.
- Active speaker indicator must be visible.

4.4 Orientation and Recording Requirements

- System must force 16:9 landscape orientation on iOS + Android, regardless of device orientation.
- Recording begins only when Host or Producer presses record.
- Multi-track recording must be enabled for Live Room and Episode 2 Room.

5. Admin Dashboard (Internal Operations System)

Goal: Provide a secure internal interface for operations: scheduling, viewing AI-created matches, managing episode lifecycles, reviewing surveys, and triggering system notifications. Dashboard is not public.

5.1 Security and Access

- Dashboard must be accessible only to authorized Admin users.

5.2 Matching Visibility (Admin Sees All AI Matches)

- Admin must be able to view all matches the AI has created, including Spotlight, Sparks.

5.3 Scheduling Workflow

- A human (Admin) coordinates an agreed upon time with participants externally (manual coordination).
- Admin sets the official Episode 1 and Episode 2 recording date/time inside the dashboard calendar.
- Once the Admin schedules the episode in the dashboard, the system must automatically send scheduling confirmations to each participant.
- System must automatically send two reminders for each scheduled episode: (1) 1 week before, (2) 48–24 hours before.

PodLove Final Specification

5.4 Surveys and Outcomes

- Dashboard must allow Admin to view survey completion status and review responses.
- Dashboard must capture and display Episode 2 outcome for the chosen pair (for AI learning loop).

5.5 QA / Bug Workflow Support

- Admin must be able to view reported system error from users

6. Chat System (Rules, Timers, and Entitlements)

Goal: Enforce correct chat access between Spotlight and chosen Spark only, with plan-based timers and non-bypassable expiration. Chat must be consistent across Mobile and Web and enforced server-side.

6.1 Chat Existence Rules

- Chat opens only **AFTER** Episode 1 recording is complete.
- Chat only exists between Spotlight and the chosen Spark.
- No global chat.
- Sparks cannot chat with other Sparks.
- Chat cannot be manually reopened once expired; expiration is automatic and enforced server-side.
- Timers must be accurate to the hour.

6.2 Chat Duration by Plan

- Sampler (Free): 72-hour chat window **if** matched (Spotlight + chosen Spark).
- Seeker: 7-day chat window.
- Scout: Unlimited chat access to matches.

6.3 Second-Chance / Refresh Implications

- Second-chance and refresh entitlements apply only to Seeker and Scout as specified in Subscription Tiers.
- Chat access must track the active chosen pair and must **NOT** open with unchosen Sparks.

6.4 Notifications Related to Chat

- System must notify both users when chat opens.
- System must notify when chat is about to close (for timed plans) and when it closes.

PodLove Final Specification

Appendix A – Subscription Tiers (Authoritative Entitlements Summary)

- Sampler (Free): 2 Sparks; 1 Spotlight appearance per residency; 72-hour chat window if matched; no second-chance as Spotlight; can re-enter pool as Spark; no bio preview.
- Seeker ($29.99/mo): 3 Sparks; 2 Spotlight appearances; 7-day chat; 1 second-chance match; no bio preview.
- Scout ($14.99/mo): 4 Sparks; 3 Spotlight appearances; unlimited chat; 2 match refresh attempts; bio preview (no photos); early access + merch discounts.
- All entitlements must be enforced automatically by backend logic.